FINAL PROJECT IN CC105-INFORMATION MANAGEMENT

GROUP 4: SYNTACTIC SYNDICATE

BADONG, JAMIELA B.
ABABA, RECHELLE ANN D.
MALCO, ROMEO JR M.
INOCENTES, MAY ANN J.
LOBERINO, JOHN NICK
BARAERO, MAY ANGEL T.
IBARRETA, JHARREL H.

MR. MICHAEL BAÑARIA
INSTRUCTOR

SYSTEM OVERVIEW

The system is structured to manage integrated student enrollments, departmental course catalogs, academic grading metrics, and user authentication logs.

Database architecture built across six normalized tables such as (DEPARTMENTS, COURSE, USERS, LOGIN_STATUS, ENROLLMENTS, and ACADEMIC_RECORDS) that linked through precise relational primary and foreign keys.

So, the Normalization Fully compliant up to the Third Normal Form (3NF), ensuring that atomic values are maintained, partial functional dependencies are eliminated via single-column primary keys, and transitive dependencies are removed.

SYSTEM OBJECTIVES

Eliminate Data Redundancy: Structure institutional information logically to prevent duplicate entries and maintain high data integrity across departments and student records.

Secure User Management: Isolate user authentication credentials (usernames and password hashes) within dedicated security tables (LOGIN_STATUS) to protect system access.

Streamline Academic Tracking: Decouple operational registration activities (ENROLLMENTS) from performance evaluation metrics (ACADEMIC_RECORDS) for cleaner grade reporting and monitoring.

Ensure Structural Efficiency: Provide a scalable database foundation capable of handling institutional transaction loads reliably while adhering strictly to relational database design standards.

Core Entities & Purpose

The schema is organized into six interconnected tables that manage institutional departments, course offerings, user accounts, enrollments, and academic performance:

DEPARTMENTS: Stores information about college departments using department_id as the Primary Key (PK).

COURSE: Manages specific subjects offered by departments, linking back via department_id (Foreign Key / FK).

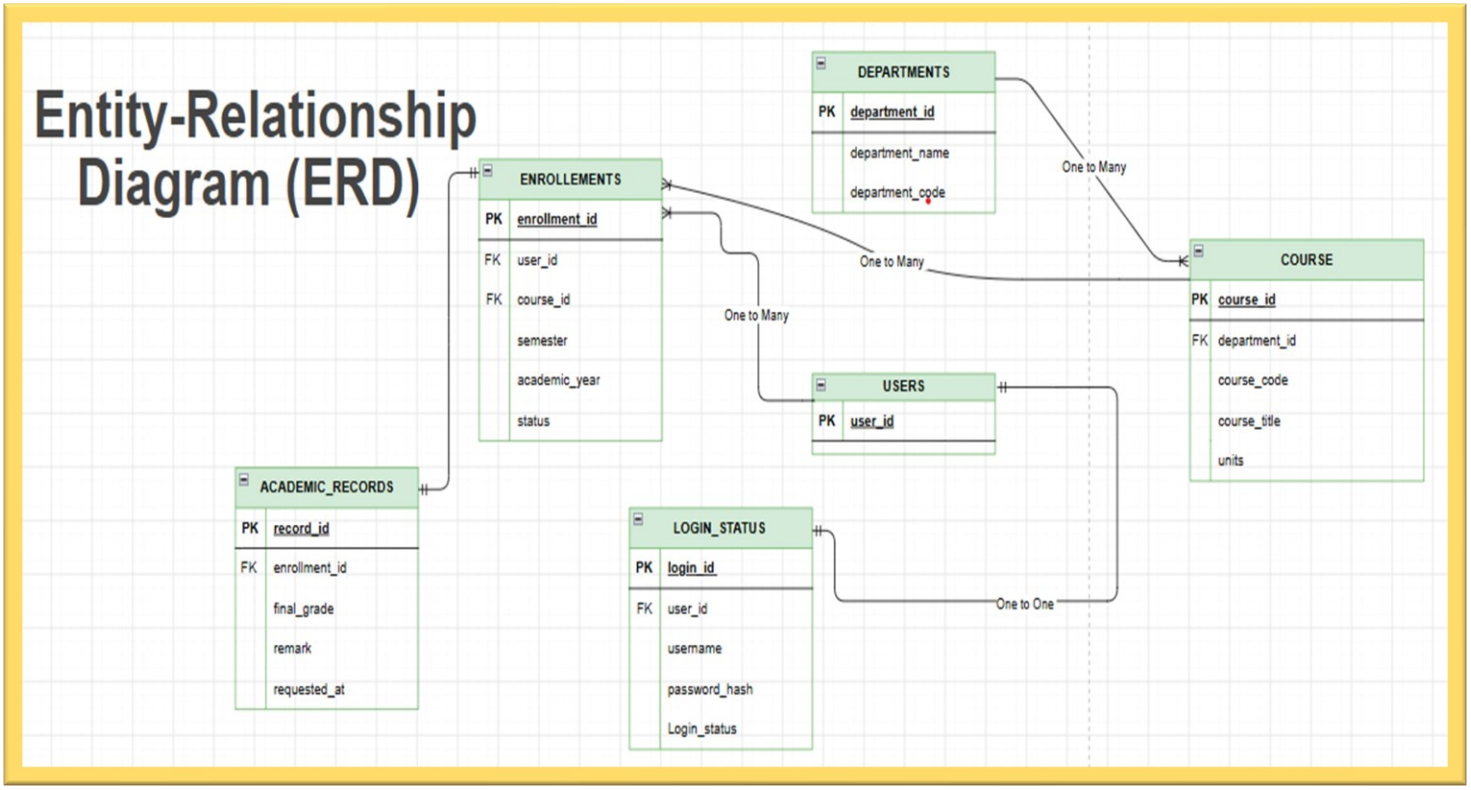
USERS: Holds profile or user data for individuals within the academic system, identified by user_id (PK).

LOGIN_STATUS: Tracks authentication details (username, password hash, and login status) mapped specifically to user accounts.

ENROLLMENTS: Acts as a junction or transaction table connecting USERS (students) and COURSE offerings for specific semesters and academic years.

ACADEMIC_RECORDS: Records final grades, remarks, and request logs tied directly to student enrollments.

ENTITY RELATIONSHIP DIAGRAM (ERD)



KEY RELATIONSHIPS & CARDINALITY

The database uses relational constraints to ensure structural integrity:

Departments to Courses (One-to-Many): A single department can offer multiple courses, but each course belongs to only one department (department_id).

Users to Login Status (One-to-One): Each user account corresponds to exactly one login credential record (user_id).

Users to Enrollments (One-to-Many): A user can enroll in multiple courses across different terms, while each enrollment record belongs to a single user.

Courses to Enrollments (One-to-Many): A course can have many student enrollments across different academic periods.

Enrollments to Academic Records: Links a specific enrollment transaction to its corresponding grades and performance remarks

DATA DICTIONARY

DEPARTMENTs Table

Column Name	Data Type	Key Type	Description
department_id	INT	**PK**	Unique identifier for the department.
department_name	VARCHAR		Full name of the department.
department_code	VARCHAR		Short code for the department.

COURSE Table

Column Name	Data Type	Key Type	Description
course_id	INT	**PK**	Unique identifier for the course.
department_id	INT	**FK**	References the department offering the course.
course_code	VARCHAR		Official course code (e.g., CC 106).
course_title	VARCHAR		Descriptive name of the course.
units	INT/DECIMAL		Credit units assigned to the course.

USERs Table

Column Name	Data Type	Key Type	Description
user_id	INT	**PK**	Unique identifier for the user profile.

LOGIN_STATUS Table

Column Name	Data Type	Key Type	Description
login_id	INT	**PK**	Unique identifier for the login record.
user_id	INT	**FK**	References the parent user profile.
username	VARCHAR		Unique login handle.
password_hash	VARCHAR		Securely hashed user password.
Login_status	VARCHAR/ENUM		Current login status indicator.

ENROLLMENTs Table

Column Name	Data Type	Key Type	Description
enrollment_id	INT	**PK**	Unique identifier for the enrollment transaction.
user_id	INT	**FK**	References the enrolled user/student.
course_id	INT	**FK**	References the enrolled course.
semester	VARCHAR		Academic semester of registration.
academic_year	VARCHAR		Academic year period.
status	VARCHAR/ENUM		Current status of the enrollment.

ACADEMIC_RECORDs Table

Column Name	Data Type	Key Type	Description
record_id	INT	**PK**	Unique identifier for the academic record.
enrollment_id	INT	**FK**	References the specific enrollment instance.
final_grade	DECIMAL		Recorded final grade score.
remark	VARCHAR		Standing remarks (e.g., Passed, Failed).
requested_at	TIMESTAMP		Timestamp of grade release or record request.

Database Normalization Proof (3NF)

This database structure complies fully up to the Third Normal Form (3NF) to ensure optimal efficiency and data integrity

1. First Normal Form (1NF):

- All tables contain only atomic values with no repeating groups or multi-valued fields. Each attribute holds a single value, and every table has a defined primary key (department_id, course_id, user_id, login_id, enrollment_id, and record_id).

2. Second Normal Form (2NF):

- The database meets 1NF criteria. Furthermore, all tables utilize single-column primary keys, completely eliminating any possibility of partial functional dependencies where a non-key attribute depends on only part of a composite key.

3. Third Normal Form (3NF):

- The database meets 2NF and contains no transitive dependencies (no non-key attribute depends on another non-key attribute).
- For instance, login credentials (username, password_hash) are isolated in login_status rather than cluttering users.
- Similarly, grading details are separated into academic_records, referencing enrollments via a foreign key (enrollment_id), ensuring that operational registration data and performance evaluation metrics are cleanly decoupled.